

案例1. 电商公司销售数据分析

数据: 订单编号 月份 一级类别 二级类别 品牌 地区
 --- 2021/10/01 电子产品 智能手机 小米 重庆

平台 价格
 京东 1099

import pandas as pd

df1 = pd.read_excel('...', sheet_name='1-6月订单数据')

df1.head() } 查看数据
 df1.shape

df2 = pd.read_excel('...', sheet_name='7-12月订单数据')

df = pd.concat([df1, df2], axis=0) 合并数据

df.info() 查看数据信息 是否有空值, 数据类型是否正确

df.duplicated().sum() 查看重复值数量

去重 { df.drop_duplicates(inplace=True) 删除重复值, 只保留第一条, 原地修改
 df.shape 观察数据量变化, 原来 20006 变成了 20001

df.isna().sum() 查看空值数

去空 { df.dropna(inplace=True) 去空, 原地修改

df.shape 数据量从 20001 到 19985

df['价格'] = df['价格'].apply(lambda x: float(str(x).replace(',', '')))

df.info() 修改价格数据类型为 float

使用 pyecharts 画各省销售地图

from pyecharts.charts import Map, Pie

import pyecharts.options as opts

→ 统计得到每个不同地区值出现次数

province = df['地区'].value_counts().sort_values(ascending=False)

或者 = df['地区'].value_counts(ascending=True)

得到的 province 是以每个地区为索引, 值为出现次数的 Series

data = list(zip(province.index, province))

也可用 province.values

得到的 data 形式如下: [('广东', 2103),

('上海', 2000),

...]

index	value
广东	2103
...	...

注意地图中市名命名问题

地图

```
m1 = Map()  # 创建地图对象
m1.add('销售号', data, 'china')  # 地图视觉映射
m1.set_global_opts(visualmap_opts=opts.VisualMapOpts(is_show=True, min=0, max=200000000),
                    toolbox_opts=opts.ToolboxOpts(is_show=True),
                    title_opts=opts.TitleOpts(title='各品牌销量', subtitle='2024年1-12月'),
                    legend_opts=opts.LegendOpts(is_show=False))
m1.render_notebook()
```

绘制各品牌销量的占比环形图

```
brand = df['品牌'].value_counts().sort_values(ascending=False)
brand_data = list(zip(brand.index, brand))
p1 = Pie()
p1.add('品牌销量', brand_data, radius=[55%, 80%])  # 内半径 外半径
p1.set_global_opts(title_opts=opts.TitleOpts(title='品牌销量占比', subtitle='2024年1-12月'),
                    legend_opts=opts.LegendOpts(orient='vertical', pos='left', pos_top='middle'))
p1.set_series_opts(label_opts=opts.LabelOpts(formatter='{b}:{d}%'))
p1.render_notebook()
```

↑ 标签名
如'小米' {c}为值
 {d}%为百分比

知识点:

pyecharts 绘图步骤:

1. 初始化图表, 可设置图表大小, 主题等 `bar = Bar(...)`
2. 使用 add 方法添加数据
3. 使用 set 方法设置各类配置项 (标题, 图例, 颜色, 动画等)
 - (1) `set_global_opts` 通用配置
 - (2) `set_series_opts` 系列配置
4. 保存或显示图表
 - (1) `render` 方法保存成 html
 - (2) `render_notebook` 方法在 jupyter notebook 中显示

1. 初始化

`bar = Bar (init_opts = opts.InitOpts (width, height, chart_id, page_title, theme, bgcolor, animation_opts))`

△ width, height: 宽, 高, 默认 900px × 500px

chart_id: 图表标识

page_title: 生成 html 网页时的标题

theme: 图表主题, 'white', 'dark', 'light', 'roma', 'romantic' 等

bgcolor: 图表背景色

animation_opts: 配置动画效果

`opts.AnimationOpts (animation_duration = 1200, 动画时长`

`animation_easing = 'elasticOut') 缓动效果, 弹性弹出`

2. 添加数据

柱形图 `... .add_xaxis(['1月', '2月', '3月'...])`

Bar `... .add_yaxis('线上', [13000, 12000, 15000, ...])` 第一系列柱子

`... .add_yaxis('线下', [-, -, -...])` 第二系列柱子

地图

Map `... .add('门店数量', data, 'china')` `maptype = 'china'` 中国地图

中国地图

`= 'world'` 世界地图

`= '山东'` 山东地图

门店数量

如: `[( '山东', 100), ('上海', 150) ... ]`

门店数量

3. set 设置配置项

3.1 例. `bar.set_global_opts` (标题配置, 图例, 工具箱, 坐标轴...
视觉映射... 等)

(1) 标题配置:

`title_opts = opts.TitleOpts` (主标题, 副标题, 位置 (可用 left, center, right), 间隔 (可用 top, middle, bottom), 标题文本样式配置, 副标题文本样式配置)
主副标题 间隔

(2) 图例配置:

`legend_opts = opts.LegendOpts` (是否显示, 位置 (可用 left, right, top, bottom), 图例方向 (可用 orient), 图例间隔, 图例文本样式配置)
图例方向 图例间隔

(3) 工具箱配置:

`toolbox_opts = opts.ToolboxOpts` (是否显示, 方向 (可用 orient), 项大小 (可用 item_size), 项间隔 (可用 item_gap), 位置 (可用 left, right, top, bottom))

共同项: `pos-left`

可以用具体数值如 `pos-left=20` (像素单位)

也可用字符串如: `pos-left='center'` `pos-top='top'`

也可用百分比如: `pos-left='85%'`

(4) 坐标轴配置:

`xaxis_opts = opts.AxisOpts` (名称 (可用 name), 是否显示 (可用 is-show), 名称位置 (可用 name-location), 名称间隔 (可用 name-gap), 名称旋转 (可用 name-rotate), 名称文本样式配置, 轴线条样式 (可用 axisline-style), 轴刻度配置 (可用 axistick-opts), 轴点间隔配置 (可用 axispoint-opts))
名称 名称位置 名称间隔 名称旋转
轴: 'center' 或 'end'
轴: 'middle' 或 'start'

(5) 视觉映射配置:

`visualmap_opts = opts.VisualMapOpts` (是否显示 (可用 is-show), 最小值 (可用 min), 最大值 (可用 max), 位置 (可用 left, right, top, bottom), 是否分段 (可用 is-pieewise), 文本样式配置 (可用 text-style-opts))
是否分段 默认分5段

3.2 例 bar.set-series-opts (标签配置, 文字样式, 线样式, 区域填充等)

(1) 标签配置:

label-opts = opts.LabelOpts (is_show, position, color, distance, font_size, rotate)

位置
{ top
left
right
bottom }

标签颜色

字体大小

角度

距图形距离

(2) 文字样式配置

textstyle-opts = opts.TextStyleOpts (color, font_style, font_weight, font_size)

颜色

正/斜
{ 'normal'
'italic' }

粗细
{ 'normal'
'bold'
'bolder'
'lighter' }

大小

(3) 线样式配置

linestyle-opts = opts.LineStyleOpts (is_show, width, opacity, type, color)

透明度
0 到 1
0 为透明

类型
{ solid
dashed
dotted }

(4) 区域填充配置

areastyle-opts = opts.AreaStyleOpts (opacity, color)

案例1, 续

并绘制各月份销售量(折线图), 销售金额(柱状图), 层叠显示

```
df-month = df.groupby('月份').agg({'订单编号': 'count', '价格': 'sum'})
                .reset_index()
```

```
df-month.columns = ['月份', '销售量', '销售金额']
```

```
df-month['月份'] = df-month['月份'].map(lambda x: x[5:7])
```

```
df-month['销售量'].min() 查看销售最小值
```

```
                .max() 大
```

```
bar = Bar()
```

```
bar.add_xaxis(list(df-month['月份']))
```

```
bar.add_yaxis('销售金额', list(df-month['销售金额']))
```

```
bar.set_global_opts(title_opts=opts.TitleOpts(title='...', subtitle='...'),
```

```
                    yaxis_opts=opts.AxisOpts(name='销售金额(元)'),
```

```
                    xaxis_opts=opts.AxisOpts(name='月份', name_location='center'))
```

```
bar.extend_axis(yaxis=opts.AxisOpts(name='销售量(台)', min_=1000, max_=2500,
    拓展一条轴
    interval=500))
```

```
line = Line()
```

```
line.add_xaxis(list(df-month['月份']))
```

```
line.add_yaxis('销售量', list(df-month['销售量']), z=2, yaxis_index=1)
```

```
line.set_series_opts(label_opts=opts.LabelOpts(is_show=False),
```

```
                    linestyle_opts=opts.LineStyleOpts(width=2))
```

```
bar.overlap(line)
```

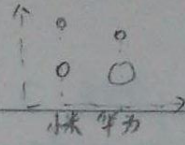
```
bar.render_notebook()
```

控制前后顺序, z值大在上, 柱形图z=1



表示使用第2条y轴, 默认

折线图不写标签



并不同品牌各价位销量气泡图

并 使用 matplotlib 绘制

```
scatter = df.groupby(['品牌', '价格'], agg({'月份': 'count'}))
    .rename(columns={'月份': '订单数'})
    .reset_index()
```

得到的 scatter 例:

品牌	价格	订单数
--	--	--
--	--	--

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(14, 8))
```

```
plt.scatter(x=scatter['品牌'], y=scatter['价格'], s=scatter['订单数'],
            alpha=0.8, c=range(169))
```

颜色, 数据集共 169 条记录

```
plt.title('不同品牌各价位销量散点图', fontsize=20)
```

```
plt.xlabel('品牌')
```

```
plt.ylabel('销售价格(元)')
```

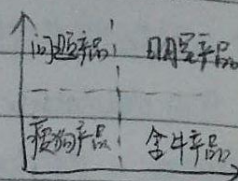
```
plt.grid(alpha=0.3)
```

```
plt.show()
```


案例2

案例2: 线下店铺产品分析-波士顿矩阵

销售增长率



数据内容:

序号	产品名称	上期销售额	本期	市场销售总额
1	开心果	¥198,655.00	¥214,561.00	¥1,654,751.00
2	和田大枣	-	-	-

市场占有率

(11行5列)

并读取数据

```
import pandas as pd
```

```
df = pd.read_excel('...')
```

df.info() 观察数据信息, 发现销售额均为字符串

```
df['上期销售额'] = df['上期销售额'].map(lambda x: float(x.replace('¥', '')))
```

```
df['本期'] = ...
```

```
df['市场销售总额'] = ...
```

```
df['销售额增长率'] = (df['本期'] - df['上期']) / df['上期']
```

```
df['市场占有率'] = df['本期'] / df['市场总']
```

并 使用 pyecharts 画图, 销售额柱形图

```
product_name = list(df['产品名称'])
```

```
sales_benqi = list(df['本期'])
```

```
sales_shangqi = list(df['上期'])
```

```
from pyecharts.charts import Bar
```

```
import pyecharts.options as opts
```

```
bar = Bar(init_opts=opts.InitOpts(height='500px', width='1200px'))
```

```
bar.add_xaxis(product_name)
```

```
bar.add_yaxis('本期销售额', sales_benqi)
```

```
bar.add_yaxis('上期', sales_shangqi)
```

```
bar.set_series_opts(label_opts=opts.LabelOpts(rotate=0))
```

```
bar.set_global_opts(xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=15))
```

```
title_opts=opts.TitleOpts(title='各商品销售额'))
```

```
bar.render_notebook()
```


C: 颜色 (支持字符串格式)

支持十六进制 "#FF0000"

支持RGB (0.5, 0.2, 0.1)

使用matplotlib 绘制 Boston 矩阵

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
plt.figure(figsize=(16,10))
```

```
plt.scatter(x=df['市场占有率'], y=df['销售额增长率'], s=df['本期销售额']/40,
```

```
c=np.random.rand(11,3), alpha=0.3)
```

```
plt.axhline(0.3, ls='--') 横线
```

```
plt.axvline(0.1, ls='--') 竖线
```

```
for x,y,z,u in zip(df['市场占有率'], df['销售额增长率'], df['产品名称'], df['市场占有率']):
```

```
    plt.text(x,y,z, fontsize=13, color='k', alpha=0.9, ha='center')
```

```
    plt.text(x,y+0.02, "%1.2f%%"%(100*u), fontsize=13, color='k', alpha=0.9, ha='left')
```

```
for a,b,c in zip((0.0,0.227,0.227), (0.01,0.69,0.01,0.69), ('宠物...', '问题...', '金牛...', '明星...')):
```

```
    plt.text(a,b,c, fontsize=20, color='k')
```

```
plt.axis([0,0.25,0,0.7]) 横纵轴坐标范围
```

```
plt.title('Boston模型', fontsize=40)
```

```
plt.xlabel('市场占有率', fontsize=20)
```

```
plt.ylabel('销售额增长率', fontsize=20)
```

```
plt.show()
```


案例3

案例3: 多元线性回归 中国GDP预测

年份	社会消费品零售总额(亿元)	总人口(万人)	GDP(亿元)
1978	1558.6	962590	3678.7
:	:	:	:
2021	--	--	--

X y

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.linear_model import LinearRegression
```

回归模型

```
from sklearn.model_selection import train_test_split
```

数据划分

```
from sklearn.metrics import mean_squared_error
```

均方误差mse

```
from sklearn.metrics import r2_score
```

R²指标

```
pd.set_option('float_format', lambda x: '%.3f' % x)
```

pandas 保留三位小数

```
gdp = pd.read_excel('r' - ...)
```

```
gdp.head()
```

```
gdp.shape
```

```
gdp.info()
```

```
gdp.isna().sum()
```

```
gdp.duplicated().sum
```

```
gdp.describe()
```

查看数据基本信息

是否有空值, 重复值

```
# 绘制社会消费品零售总额, 总人口, GDP 三列箱线图
```

```
gdp.iloc[:, 1:].plot(kind='box')
```

```
plt.grid()
```


使用线性回归建模, 社会消费品零售总额和总人口为 X, GDP 为 y

X = gdp.iloc[:, 1:-1]

y = gdp.iloc[:, -1]

拆分数据集

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=0)

构建线性回归模型, 使用训练集训练

lr = LinearRegression(fit_intercept=False)

lr.fit(X_train, y_train)

print(lr.coef)

训练好的模型参数, 分别为参数 a 和 b

即 $GDP = a \times \text{社会消费品零售总额} + b$

$lr.coef[0]$

$lr.coef[1]$

使用训练好的模型计算 X_train 对应的 GDP

y_train_pred = lr.predict(X_train)

即 y_train 为真实值, y_train_pred 为模

查看均方误差 mse, 均方根误差 RMSE, R2 分数

型结果

MSE = mean_squared_error(y_train, y_train_pred)

RMSE = MSE ** 0.5

R2 = r2_score(y_train, y_train_pred)

使用 plt 画折线图对比真实值和预测值

plt.figure(figsize=(20, 6))

train_num = X_train.shape[0]

plt.plot(range(train_num), y_train, marker='o', label='GDP-真实(亿元)')

plt.plot(range(train_num), y_train_pred, marker='>', label='预测')

plt.xticks(range(train_num), list(range(train_num)))

plt.title('训练集实际 GDP & 预测 GDP 折线图')

plt.xlabel('年份')

plt.ylabel('金额')

plt.legend()

plt.grid()

plt.show()

测试集效果查看

`y_test_pred = lr.predict(X_test)`

`MSE_test = mean_squared_error(y_test, y_test_pred)`

`RMSE_test = MSE * 0.5`

`r2_test = r2_score(y_test, y_test_pred)`

`test_num = X_test.shape[0]`

`plt.figure(figsize=(20,6))`

`plt.plot(range(test_num), y_test, label='GDP-真实(亿元)')`

`plt.plot(range(test_num), y_test_pred, label='GDP-预测(亿元)')`

`plt.xticks(range(test_num), list(range(test_num)))`

标注极值

`for x, y in zip(range(test_num), y_test_pred):`

`plt.text(x, y+30000, round(y,2))`

`for x, y in zip(range(test_num), y_test):`

`plt.text(x, y, round(y,2))`

`plt.title('测试集实际GDP & 预测GDP折线图')`

`plt.xlabel('年份')`

`plt.ylabel('金额')`

`plt.legend()`

`plt.grid()`

`plt.show()`

#多项式拟合测试

```
from sklearn.preprocessing import PolynomialFeatures
```

```
x = gdp.iloc[:, 1:-1]
```

```
y = gdp.iloc[:, -1]
```

```
lst = []
```

```
for i in range(1, 51):
```

```
    lr = LinearRegression(fit_intercept=False)
```

```
    pl = PolynomialFeatures(degree=i)
```

```
    lr.fit(pl.fit_transform(x), y)
```

```
    y_pred = lr.predict(pl.fit_transform(x))
```

```
    lst.append(mean_squared_error(y, y_pred) * 0.5)
```

```
plt.plot(range(1, 51), lst, marker='o')
```

```
plt.grid()
```

```
plt.title('1-50次函数拟合均方根误差散点图')
```

```
plt.xlabel('函数次数') 
```

```
plt.ylabel('均方根误差')
```

```
plt.show()
```

```
print(f'从RMSE看, {np.argmin(lst)}次函数拟合效果最好!')
```

lst中最小值对应的索引

市场类指标: 行业销售量, 行业销售总额, 市场增长率, 竞争对手销售总额
成交转化率 = 成交量 / 访问量

SWOT分析: S优势 W劣势 O机遇 T挑战

PEST分析: P政治 E经济 S社会 T技术

波特五力: 供应商讨价还价力, 购买者讨价还价力, 潜在竞争者进入能力, 替代品替代力, 行业内竞争者竞争力

python变量 随时命名, 随时赋初值, 随时使用
(使用前需赋初值)

python中注释不被解释器识别和运行

python中变量可代表任意数据类型

python中数值类型包括: int, float, complex, bool

python查看帮助 str? help(str)

jupyter中的

r.content 为HTTP响应的二进制形式, 使用 r.content.decode() 解码

常用于网页解析的第三方库: re, 正则表达式, lxml, BeautifulSoup4

ndarray.ndim 秩

---- .shape 数组维度 (n,m) n行, m列

---- .size 元素总个数 n*m

---- .itemsize 每个元素的字节数

n1 = np.array([1, 2])

n2 = np.array([3, 4])

n1 * n2 为 [1³ 2⁴]

获得DataFrame数据形状用 df.shape 不是 df.shape()

`pandas.read_excel(io, sheet_name=0, header=0, names=None, index_col=None, usecols=None, dtype=None, engine=None, converters=None, skiprows=None, na_values=None, thousands=None, comment=None, skipfooter=0, convert_float=True, **kwargs)`

`io`: 文件路径 'r' - ...

`sheet_name`: 导入的 sheet 页, 0 表示第一页

`sheet_name='表名'`

`sheet_name='SheetN'` 第N个 sheet, S 需大写

`sheet_name=None` 导入所有页, 返回数据类型的列表

`header`: 用哪一行作列名, 默认为 0, 第 1 行

`header=[0, 1]` 用前两行作列名

`names`: 自定义列名 如: `names=['销量', '单价', '总额']`

适用于缺少列名情况, `names` 长度必须和 Excel 列长度一致

`index_col`: 用作索引的列 默认不带行索引, 自动从 0 开始索引

`index_col=0`, 以第一列作行索引

`index_col=[0, 1]` 以前两列作为行索引

`usecols`: 需读哪些列, 默认所有列

`usecols=[0, 2, 3]`

`usecols=['年', '月']`

`converters`: 强制规定列的类型

`converters={'时间': str, '单价': float}`

DataFrame

`DataFrame.to_excel(excel_writer, sheet_name='Sheet1', na_rep='',
float_format=None, columns=None, header=True, index=True,
index_label=None, startrow=0, engine=None, encoding=None, verbose=True)`
excel_writer: 文件路径 r'...'
sheet_name: 导出表格名
index: 是否写入索引

追加写入Excel表

```
writer = pd.ExcelWriter('数据.xlsx', mode='a')  
df.to_excel(writer, sheet_name='sheet2')  
writer.save()  
writer.close()
```

或 `with pd.ExcelWriter('...xlsx', mode='a') as writer:`
`df.to_excel(writer, sheet_name='sheet2')`

直接筛选

`df['单价']` 一列

`df[['单价', '销量']]` 两列

`df[1:3]` 第1行和第2行 `df[1:2]` 第1行

条件筛选

`df[df['门店'] == '门店3']`

& 与 ~ 非

`df[(df['收入'] >= 4000) & (df['成本'] >= 3000)]`

loc索引器

iloc索引器, 前闭后开

`df.loc['行']`

`df.iloc[3, 1:6]`

`df.loc['行1': '行2', ['列1', '列2']]`

`df.loc['行1': '行2', '列1': '列2']`

`df.loc[df['列'] > 条件, ['列1', '列2']]`

使用 xpath 爬取数据基本流程

```
import requests
```

```
from lxml import etree
```

```
url = "要爬取的网页地址"
```

```
headers = { ... } 反爬虫设置
```

```
resp = requests.get(url, headers=headers)
```

```
html = resp.content.decode()
```

```
element = etree.HTML(html)
```

```
name = element.xpath("xpath Helper 提取出的路径编码 / text()")
```


商务数据分析及应用

1. 数据分类

(1) 定类数据：类别，不能计算，如红色，黄色

(2) 定序数据：分类+顺序，如小学，初中，高中，大学

(3) 定距数据：数值，没有绝对零点

(4) 定比数据：数值，有绝对零点

2. 大数据 4V: Volume, Velocity, Value, variety

3. 商务数据特征：样本量大，关联性强，时效快

--- 分类：数字类，图形类，图表类

4. 商务数据分析原则：

目的性原则，系统性，针对性，简约性

5. 商务数据分析方法：

(1) 常规分析：呈现原始数据

(2) 统计推断分析：基于原始数据推导和挖掘变化趋势

(3) 自建模型分析：要求最高，最能挖掘出价值的

6. 常用数据分析法：

(1) 对比分析：同比，环比

(2) 二八定律：80/20法则，少数关键法则

(3) 聚类分析：群分析，常见应用为用户画像分析，细分市场，研究用户行为

(4) 漏斗分析：流程分类法，常用于分析转化效果

(5) 描述统计分析法：平均数，标准差，中位数，众数

(6) 相关分析：两个或两个以上变量的相关程度，可以用散点图

(7) 回归分析：建立回归方程，进行因果分析

(8) 七何分析：5W2H分析法 ^{who} when where what why How, Howmuch

(9) 标杆分析法

7. 商务数据分析指标

(1) 流量类指标

浏览量: 页面被浏览次数

访客数: 多少人访问此网页 (去重)

跳出率: $\text{浏览量为1的访客数} / \text{总访客数}$

人均浏览量: $\text{总浏览量} / \text{总访客数}$

平均停留时间: $\text{访问停留总时长} / \text{总访客数}$

点击数, 点击人数

点击转化率: $\text{点击数} / \text{浏览量}$

老访客数, 新访客数

(2) 交易类指标

下单买家数, 下单金额, 支付买家数, 支付金额

客单价 = $\text{总支付金额} / \text{支付买家数}$

下单转化率 = $\text{下单买家数} / \text{总访客数}$

支付转化率 = $\text{支付买家数} / \text{总访客数}$

UV价值 = $\text{总支付金额} / \text{总访客数}$

平均每位客户的支付金额

(3) 商品类指标

支付商品数, 下单商品数, 商品动销率 = $\text{支付商品数} / \text{店铺在售商品数}$

收藏人数, 加购人数

(4) 其他指标

成功退款金额, 评价数, 有用评价数, 正面评价数, 负面..., 投诉已结案数, 发货数

8. 商务数据分析流程

明确需求 → 采集数据 → 处理数据 → 分析数据 → 呈现数据 → 撰写报告

9. 数据清洗工具: 记事本, Excel, VBA, Python

10. 清洗商务数据方法

(1) 修复缺失值

(3) 修复逻辑错误

(2) 修复错误值

(Excel中, IFERROR函数)

(4) 统一数据格式

(5) 删除重复值

Date. /

市场和行业数据分析

11. 市场容易体现的是市场规模大小

支付金额较父行业占比指数越高, 该品类市场容易占比越大

12. 市场变化趋势可理解为市场生命周期

企业在市场不同阶段采取相应的运营策略

导入期 → 成长期 → 成熟期 → 衰退期

快速占领市场

提供售后服务

准备退出市场

扼杀市场

13. 市场潜力可借助蛋糕指数和市场容量来分析

蛋糕指数 = 市场容量 / 竞争对手数

= 支付金额较父行业占比指数 / 父行业卖家数占比

蓝海: 需求旺盛, 竞争小

红海: 竞争极端激烈

14. 行业集中度可以反映行业饱和度和垄断程度

赫芬达尔指数 = 主要竞争对手市场占有率平方值求和

指数越大, 行业集中度越大, 数值趋近于1时, 出现垄断时

指数的倒数表示本行业市场份额被多少品牌瓜分。

15. 行业稳定性

波动系数 = 标准差 / 平均值 越小越稳定

极差 = 最大 - 最小

16. 行业前景可用波士顿矩阵分析法

销售增长率 ↑ 问题商品 畅销商品

瘦狗商品 金牛商品 市场占有率

竞争对手数据分析

1. 竞争对手分类:
- 与企业竞争相同市场的长期固定竞争对手
 - 在经营活动中与企业有局部竞争的局部竞争对手
 - 就某一事件与企业有竞争的暂时性竞争对手

2. 竞争对手的识别:

市 场 类 型	完全竞争市场:	许多交易相同商品的企业和客户
	垄断竞争市场:	许多出售相似但不完全相同商品的企业
	寡头市场:	只有几个提供相似或相同商品的企业
	垄断市场:	只有一个提供商品的企业

4个原则识别竞争对手

- (1) 提供相同或类似的商品或服务
- (2) 具有共同或基本重合的市场范围
- (3) 具有基本相同的客户定位, 客户可以互换
- (4) 在特定时间内争夺排他性市场资源

3. 分析品牌潜力

高增长 (交易增长幅度高), 高销量

4. 分析竞争对手数据

动销率: 有销量的商品品种数 / 商品品种总数, 反映进货品种的有效性

售罄率: 销售数量 / 进货数量, 反映商品销售速度, 商品受欢迎程度

< 65%: 库存大量积压, > 85%: 脱销

5. SWOT分析

S: 优势 W: 劣势 O: 机会 T: 威胁

	S	W
O	杠杆效应,	抑制性 (机会和劣势不匹配)
T	脆弱性	问题性 (平喘挑战)
	(环境威胁, 优势得不到发挥)	

1. 商品分析的主要内容: 流量分析, 定价分析, 库存分析

2. 商品定位:

引流型商品: 价格低, 利润少

活动型商品: 清库存, 提销量, 品牌宣传, 利润最低

利润型商品: 销量不一定最好, 利润高, 适用小众人群

形象型商品: 高品质, 高客单价, 极小众

边缘型商品: 完善商品布局, 提升好感

3. 分析商品流量结构 (分析下单渠道)

4. 分析页面流量 (浏览量, 访客数, 浏览量为1的访客数, 平均停留时间, 跳失率)

跳失率 = 浏览量为1的访客数 / 访客数

5. 分析商品关键词

核心词: 牛仔裙

修饰词: 男, 宽松, 直筒

长尾词: 核心词和修饰词组成

品牌词

组合标题时, 尽量将引流能力强的词放最前或最后, 至少包含两个精准核心词

可以用竞争度来判断标题是否精准匹配客户搜索

竞争度 = 成交量 / 在线商品数 = (搜索人气 × 点击率 × 支付转化率 / 在线商品数) × 100

6. 商品定价方法:

(1) 成本毛利法: 目标定价 = 成本 × (1 + 目标毛利率)

例: $280 = 200 \times (1 + 40\%)$

(2) 竞品参考法: 参考竞品商品来定价, 需建立在成本毛利法基础上

(3) 客户价值法: 根据客户对商品价值的感知程度

7. 黄金价格点法定价

商品定价 = 竞争对手最低价 + (竞争对手最高价 - 最低价) × 0.618

8. 分析商品库存天数

每天卖多少件?

$$\text{库存天数} = \text{期末库存数量} \div (\text{某销售期的销售数量} \div \text{销售期天数})$$

9. 分析库存周转率

$$\text{库存周转率} = \text{销售数量} \div [(\text{期初库存量} + \text{期末库存量}) \div 2]$$

库存天数高, 库存周转率低的SKU容易库存积压

库存天数低, 库存周转率高的SKU容易断货

销售数据分析

1. 销售数据体现销售业绩，包括交易数据，推广数据，利润数据
2. 分析交易趋势：指定时期内，销量，销售额，交易客户数，客单价（销售额/客数）
环比增幅
3. 分析销售额差异（目标和实际）受销量/售价影响程度
4. 分析推广渠道，可使用投入产出比指标。
5. 分析活动推广效果

流量：活动为企业带来的流量（访客数，点击量，点击率，跳失率）

转化：流量转化情况（收藏数，加购数，成交数）

拉新：带来的新客户情况（新访客，新收藏数）

留存：活动后客户留存（会员数，交易次数）

6. 利润与利润率

利润 = 成交金额 - 总成本

销售利润率 = (利润 / 成交金额) × 100%

成本利润率 = (利润 / 总成本) × 100%

客户数据分析

1. 客户数据内容

(1) 描述性数据:

基本信息: 姓名, 性别, 年龄, 所在地, 工作类型, 职位, 收入水平, 家庭成员

行为偏好: 购物时间段, 是否爱收藏, 加购, 偏好价格, 偏好颜色, ...

(2) 交易性数据:

商品购买记录, 购买频率, 数量, 金额等

2. 客户数据价值

客户分级的依据

企业决策的基础

加强客户互动的指南

改进商品销售表现

3. 客户标签: 客户群体标记

根据客户标签, 有针对性推广商品

4. 客户生命周期: 新客户, 活跃客户, 休眠客户, 流失客户

通过上一次交易时间距今时间计算

5. 客户忠诚度: 购买次数, 重复购买率

6. 分析和挖掘客户价值

六大指标: 最近一次消费时间: 与当前时间相减, 间隔越大, 指标越小

消费频率: 指定时期内重复购买次数, 越大, 指标越高

消费金额: 越大, 指标越高

最大单笔: - - - - -

特价商品占比: - - - - -

最高单价商品占比: - - - - -

7. RFM模型 R: Recently F: Frequency M: Monetary

#4. 查看df信息

```
df.info()
```

#5. 将三列“上升幅度”、“点击率”、“转化率”中百分号删除，并转化为浮点数

```
df[['上升幅度', '点击率', '转化率']] = df[['上升幅度', '点击率', '转化率']].applymap(  
    lambda x: float(str(x).replace('%', '')) / 100)
```

#为“热度排名”列添加从1到298的连续整数

```
df['热度排名'] = range(1, 298)
```

```
df.head()
```

#6. 将除“关键词”列之外所有列转换成浮点类型

```
df1 = df.drop('关键词', axis=1).astype(float)
```

```
df1.insert(1, '关键词', df['关键词'])
```

```
else 0) df1.head()
```

#7. 绘制转化率直方图

```
df1['转化率'].hist(bins=20, figsize=(10, 4), edgecolor='red')
```

```
plt.title('转化率直方图', fontsize=20)
```

```
plt.xlabel('转化率')
```

```
plt.ylabel('出现频数')
```

```
plt.show()
```

#8. 绘制点击率和转化率的箱线图

```
df1[['点击率', '转化率']].boxplot(figsize=(12, 8))
```

```
plt.title('点击率与转化率箱线图', fontsize=20)
```

```
plt.xlabel('点击率/转化率')
```

```
plt.ylabel('指标值')
```

```
plt.show()
```


9. 绘制市场平均出价前 20 名关键词条形图

```
data = df1.sort_values('市场平均出价').tail(20)[['关键词', '市场平均出价']]
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
plt.figure(figsize=(18, 8))
```

```
plt.barh(data['关键词'], data['市场平均出价'])
```

```
plt.title('市场平均出价前 20 名关键词条形图', fontsize=20)
```

```
plt.xlabel('市场平均出价')
```

```
plt.ylabel('关键词')
```

```
plt.xticks(np.linspace(0, 1.1, 12))
```

```
plt.grid(axis='x')
```

```
plt.show()
```


1. 国家大力推进电子政务建设, 提高政务数据的科学性, 准确性, 时效性

2. 数据交易市场通过生产数据, 研发和分析数据, 为数据交易提供帮助 (X)

(数据交易市场不直接参与数据生产、研发和分析, 只是提供一个平台, 帮助数据供应商和数据需求方进行交易)

3. 大数据分析流程

明确分析目标 — 搜集所需目标 — 数据处理 — 分析挖掘 — 数据可视化 — 撰写数据报告

4. 替代品和互补品价格上涨导致本商品需求最变动分别为 90, 70,

则本商品需求变动为 $90 - 70 = 20$

5. 供应商考核标准: 产品质量, 交货及时性, 价格和成本, 供应商管理, 创新能力, 服务水平, 可持续性和社会责任, 合作关系

实验1

实验题 1:

1. 数据获取: 通过爬虫函数, 请求爬取百度首页信息, 要求返回状态码

```
import requests
```

```
response = requests.get('https://www.baidu.com')
```

```
print('当前状态码为:', response.status_code)
```

```
response.text[:100]
```

2. 数据格式化输出: 根据题目要求, 输出固定格式的字符串

A公司2022年销售收入为4056.3425万元, 要求保留两位小数

```
print('2022年销售收入为{:2f}万元'.format(4056.3425))
```

3. 函数应用: 库存管理中 期末库存 = 期初库存 + 本期入库数 - 本期出库数。

```
def 期末库存计算(期初库存, 本期入库, 本期出库):
```

```
    期末库存 = 期初库存 + 本期入库 - 本期出库
```

```
    return 期末库存
```

实验题 2

波士顿矩阵认为决定产品结构的基本因素有两个: 市场引力与企业实力

市场引力包括: 市场销售增长率, 竞争对手强弱及利润高低, 其中最关键的是销售增长率

企业实力包括市场占有率、技术、设备、资金利用能力等, 其中市场占有率是决定产品结构的内在要素

本案例对企业两年各类产品销售情况进行分析。

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import warnings
```

```
warnings.filterwarnings('ignore')
```

Date: /


```

pd.set_option('float-format', lambda x: '%.2f' % x)
from pyecharts.charts import Bar
import pyecharts.options as opts

df = pd.read_excel('r'...)
df.rename({'市场销售总额': '本期市场销售总额'}, axis=1, inplace=True)
df.head()

df = df.set_index(['序号', '产品名称']).applymap(lambda x: float(x.replace(
    ('¥', ''), replace(',',''))))

df.reset_index(inplace=True)
df.head()

df['销售增长率'] = (df['本期销售'] - df['上期销售']) / df['本期销售']
df['市场占有率'] = df['本期销售'] / df['本期市场销售总额']

```

```

product_name = df['产品名称'].to_list()
sales_benqi = df['本期销售'].to_list()
sales_shangqi = df['上期销售'].to_list()
df_sales = pd.DataFrame({'产品名称': product_name, '本期销售': sales_benqi,
    '上期销售': sales_shangqi})

```

```

df_sales.head()

bar = Bar(init_opts=opts.InitOpts(height='500px', width='1200px'))
bar.add_xaxis('product_name')
bar.add_yaxis('本期销售', sales_benqi)
bar.add_yaxis('上期销售', sales_shangqi)

bar.set_series_opts(label_opts=opts.LabelOpts(rotate=0))
bar.set_global_opts(xaxis_opts=opts.AxisOpts(name='产品名称', axislabel_opts
    = opts.LabelOpts(rotate=45)),
    yaxis_opts=opts.AxisOpts(name='金额(元)'),
    title_opts=opts.TitleOpts(title='各商品销售情况'))

```

```

bar.render_notebook()

```


001100

绘制波士顿矩阵

plt.figure(figsize=(16,10))

plt.scatter(x=df['市场占有率'], y=df['销售额增长率'], s=df['本期销售额']/40,
c=np.random.rand(11,3), alpha=0.3)

plt.axhline(0.2, ls='--')

plt.axvline(0.1, ls='--')

for x, y, z in zip(df['市场占有率'], df['销售额增长率'], df['产品名称']):

plt.text(x, y, z, fontsize=12, ha='center')

plt.text(x, y-0.01, "%1.2f%%" % (100*x), fontsize=12, ha='center')

for a, b, c in zip((0, 0, 0.227, 0.227), (0.01, 0.43, 0.01, 0.43),

('瘦狗产品', '问题产品', '金牛产品', '明星产品')):

plt.text(a, b, c, fontsize=20, color='k')

plt.axis([0, 0.25, 0, 0.45])

plt.title('产品 Boston 分析', fontsize=40)

plt.xlabel('市场占有率', fontsize=20)

plt.ylabel('销售额增长率', fontsize=20)

plt.show()

实操题3:

```
def func(P, i, n):
```

```
    list1 = []
```

```
    for k in range(1, n+1):
```

```
        F = P / (1+i)**k
```

```
        list1.append(round(F, 2))
```

```
    return list1
```

```
print('未来5年, 每年的现金流量实际购买力价值:', func(100, 0.05, 5))
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(6, 6))
```

```
plt.bar(range(5), func(100, 0.05, 5), tick_label=[f'第{i}年' for i in  
range(1, 6)], width=0.5, alpha=0.4, edgecolor='black')
```

```
plt.title('未来5年现金流量实际购买力状况图', fontsize=16)
```

```
plt.xlabel('年份')
```

```
plt.ylabel('金额')
```

```
for x, y in zip(range(5), func(100, 0.05, 5)):
```

```
    plt.text(x, y, y, ha='center', va='bottom')
```

```
plt.grid()
```

```
plt.show()
```


第四题

- ```
from pyecharts.charts import Pie
from pyecharts import options as opts
```
- △ `df = pd.read_excel('...')`  
`print(df.shape)`  
`df.head()`
- △ `df['库存数量'] = df['库存'].map(lambda x: x[-1])` <sup>int</sup>  
 #3 库存数量为新加列，去掉库存列中的单位获得  
`df.head()`
- △ `df['库存成本'] = df['库存数量'] * df['库存单价']` #4 计算库存成本  
`df.head()`
- #5 根据 '仓库名称' 列分组，汇总统计各仓库成本合计并排序降序  
 △ `warehouse_cost = df.groupby('仓库名称').agg({'库存成本': 'sum'}).sort_values('库存成本', ascending=False)`
- #6. pyecharts 绘制 pie 图 数据准备  
 # 将数据组织成 [(仓库名称, 库存成本总额), (...)]  
`data_warehouse_name = warehouse_cost.index`  
`data_warehouse = list(zip(data_warehouse_name, warehouse_cost['库存成本']))`
- #7. 绘制饼图 标题为 '库存成本占比分析', 副标题为 '2022年7月'  
`pie = Pie()`  
 △ `pie.add('仓库库存成本', data_warehouse)`  
 △ `pie.set_global_opts(title_opts=opts.TitleOpts(title='...', subtitle='...'))`  
 # 展示图形  
`pie.render_notebook()`  
 # 通过饼图可以看出，北京仓库库存成本为 一元，占总库的 % (四舍五入，保留2位小数)



## 第五题

# 导入 numpy, pandas, matplotlib

```
{ import numpy as np
 import pandas as pd
 import matplotlib.pyplot as plt
```

# 2. 读取 Excel 文件中的 '订单数据', '业务人员' sheet 表, 分别赋值给 data1, data2.

```
data1 = pd.read_excel('...', sheet_name='...')
```

```
data2 = ...
```

```
print(data1.shape)
```

```
print(data2.shape)
```

# 3. 数据合并, 使用 '地区' 作为键, 合并 data1 和 data2, 左连接, 直接后赋值给 data3.

```
data3 = pd.merge(data1, data2, how='left', on='地区')
```

```
print('合并后数据行列数分别为: {}行, {}列.'.format(data3.shape[0], data3.shape[1]))
```

```
data3.head()
```

# 4. 查看数据中所有空值的数量, 重复行的数量, 分别赋值给 nan\_num, row\_duplicated

```
nan_num = data3.isnull().sum()
```

isnull 也可用 isna

```
row_duplicated = data3.duplicated().sum()
```

# 绘制不同地区销售业绩, 利润额条形图.

# 5. 数据准备

# 根据 '省/自治区' 列进行分组, 分别对 '利润', '销售业绩' 列求和, 并根据利润额升

序排列, 得到各省市利润总额和销售总额, 赋值给 barh\_data

```
barh_data = data3.groupby('省/自治区').agg({'利润': 'sum', '销售业绩': 'sum'})
 .sort_values('利润')
```

| 省/自治区 | 利润  | 销售业绩 |
|-------|-----|------|
| 山东省   | 100 | ...  |
| 北京市   | 200 | ...  |



练习1700

6.

# 使用 barh-data 数据绘制不同地区销售总额、利润额条形图

# 画布大小 (16, 12), 柱宽为 0.8 width 为柱体宽度, linewidth 为柱体宽度

# 设置图形标题为 '各省市销售总额、利润额条形图', 字体大小为 20

# x 轴标题为 '金额(元)', y 轴标题为 '省/自治区'

# 显示网格, 显示图例

```
barh = barh-data.plot(kind='barh', figsize=(16, 12), title='...',
 xlabel='...', ylabel='...', grid=True, legend=True,
 title.set_size(20),
 width=0.8 (柱宽),
 font_size=20 (轴标签字体大小))
plt.show()
```

# 根据条形图显示: 利润额最多的省份是: \_\_\_\_, 利润额为 \_\_\_\_ 元。

亏损最多的省份: \_\_\_\_, 亏损金额为 \_\_\_\_ 元。(四舍五入, 保留小数)

# 统计不同地区经理不同类别的销售情况, 并绘制折线图, 以便了解不同专业背景人员在不同类别的表现差异, 完成销售人员的基本 KPI 考核

7. # 数据准备, 使用 pivot-table 数据透视表功能生成数据

# 其中数据来源于 data3, 行数据为 '地区经理' 列, 列数据为 '类别' 列, 统计值为 '销售总额', 统计方法为求和

# 完成数据清洗统计并赋值给 tem-data

```
tem-data = pd.pivot_table(data3, index=['地区经理'], columns=['类别'],
 values=['销售总额'], aggfunc={'销售总额': 'sum'})
```

# 8. 使用 tem-data 数据循环绘制每个地区经理各类别销售总额折线图

# 折线图标题为 '0', 线型为 '--'

# 画布大小为 (16, 7), 柱宽为 0.8 linewidth (线宽)

# 图形标题为 '地区经理各商品类别销售总额折线图', 标题字体大小为 20

# x 轴标题为 '类别', y 轴标题为 '金额(元)'

# 显示网格, 显示图例



```

△ for i in range (tem_data.shape[0]):
 tem_data[i:i+1].stack().plot(kind='line', marker='o', linestyle='--',
 figsize=(16,7), title='...', fontsize=20,
 xlabel='...', ylabel='...', grid=True, legend=True)
 .title.set_size(20)

```

# 9. 将销售额1000以上且亏损的订单筛选出来，保存至 '大额亏损订单.csv'

```

△ data4 = data3[data3['销售额'] > 1000 &

```

```

data4.to_csv('大额亏损订单.csv')

```

# 大额亏损订单中，销售额最大的订单所在地区是\_\_\_\_，利润为\_\_\_\_。

\_\_\_\_，最多的类别是\_\_\_\_，最多的地区是\_\_\_\_。

相对指标是两个绝对数之比，如经济增长率、物价指数、固定资产投资增长率等。

平均指标：人均利润

总量指标：年末人口总数

时期指标：一段时期内的总量，如产品产量、能源消耗量、财政收入、商品零售额等。